

Activity 1 : Compare Flutter, React Native, Kotlin and Swift

1.1 Overview : Strengths and Weaknesses

Flutter (Google, Dart)

- **Strengths:** Single codebase compiles to native ARM code for iOS, Android, and Web, consistent pixel perfect UI across platforms via its own rendering engine.
- **Weaknesses:** Larger app/bundle size, smaller Dart talent pool, and native module bridging can be verbose for platform specific features.

React Native (Meta, JavaScript/TypeScript)

- **Strengths:** Very large npm/JS ecosystem, huge developer talent pool, code sharing with web via React Native Web, fast iteration with Fast Refresh, mature third party SDK support.
- **Weaknesses:** Historic bridge overhead for heavy computation (mitigated by the New Architecture/Fabric and JSI), occasional need for native modules for platform specific polish.

Kotlin Multiplatform (KMP)

- **Strengths:** Shares business logic, networking, and data layers across iOS/Android while UI remains fully native, giving near native performance and platform fidelity.
- **Weaknesses:** UI is not shared (must be built twice unless using the still maturing Compose Multiplatform), smaller ecosystem, steep learning curve for iOS developers unfamiliar with Kotlin, immature web/desktop support.

Swift / SwiftUI

- **Strengths:** Best possible iOS performance and platform fidelity; first class access to Apple frameworks such as HealthKit and Core ML ideal for on device AI.
- **Weaknesses:** iOS only Android and Web require entirely separate codebases, resulting in the highest development and maintenance cost and slowest overall time to market for a multi platform product.

1.2 Comparative Analysis

The four technologies were evaluated against the criteria specified in the lab sheet: development speed, code reusability, performance, ecosystem support, learning curve, web compatibility, AI/ML integration, real time features, maintenance cost, and security.

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift / SwiftUI
Development Speed	High	High	Medium	Low (multi-platform)
Code Reusability	High (~95%)	High (~90%)	Medium (logic only, ~50%)	Very Low (0%)
Performance	High	Medium High	Very High	Very High
Ecosystem Support	Medium-High	Very High	Medium	High (iOS only)
Learning Curve	Medium (Dart)	Low-Medium (JS/TS)	High	Medium
Web Compatibility	High (Flutter Web)	High (RN Web)	Low-Medium (experimental)	None (native)
AI/ML Integration	Medium	High	Medium	Very High (Core ML)
Real-time Features	High	High	High	High
Maintenance Cost	Low	Low	Medium-High	High
Security	High	Medium High	High	Very High

Table 1.1: Comparison of Flutter, React Native, Kotlin Multiplatform, and Swift/SwiftUI

1.3 Suitability for the FitFlow Fitness App

FitFlow needs seamless experience across iOS, Android and Web High performance for real time workout tracking AI personalization camera based computer vision for nutrition logging Rapid iteration from a mid sized team under commercial pressure to reverse a declining retention trend. We exclude Swift/SwiftUI since it does not support Android or Web unless the whole codebase is duplicated, which does not align with FitFlow's requirement for rapid market launch and cost efficiency. Kotlin Multiplatform is fast, but duplicates a lot of UI work and has no mature web story, making it slower and more expensive for this project than a true cross platform UI framework. Both Flutter and React Native are good in terms of being cross platform.

1.4 Recommendation

The FitFlow redesign is recommended to use React Native (React Native Web for the web client). It provides the best combination of development speed, code sharing on all three targeted platforms, and access to the largest ecosystem of real time and AI/ML integration libraries all of which is critical given the tight timeline and mid sized

engineering team of FitFlow. React Native New Architecture (Fabric/JSI) greatly reduces the performance gap with fully native apps and this is enough for FitFlow workout tracking and FitFlow social features. A hybrid approach is best at the edges: performance isolated modules, for example the on device computer vision pipeline for nutrition logging can be made as native modules, in Kotlin or Swift if profiling shows bottlenecks. The majority of the application still stays as one React Native and TypeScript codebase. This suggestion also matches the technology direction followed in the FitFlow case study.